is therefore still a risk that an attacker could compromise these credentials and then use the KMS to decrypt the other secrets.

> **TIP** You can often restrict a KMS to only allow your keys to be used from clients connecting from a *virtual private cloud* (VPC) that you control. This makes it harder for an attacker to use compromised credentials because they can't directly connect to the KMS over the internet.

A solution to this problem is to use short-lived tokens to grant access to the KMS or secrets manager. Rather than deploying a username and password or other static credential using Kubernetes secrets, you can instead generate a temporary credential with a short expiry time. The application uses this credential to access the KMS or secrets manager at startup and decrypt the other secrets it needs to operate. If an attacker later compromises the initial token it will have expired and can't be used. For example, Hashicorp Vault (https://vaultproject.io) supports generating tokens with a limited expiry time which a client can then use to retrieve other secrets from the vault.

> **CAUTION** The techniques in this section are significantly more complex than other solutions. You should carefully weigh up the increased security against your threat model before adopting these approaches.

If you primarily use OAuth2 for access to other services, you can deploy a short-lived JWT that the service can use to obtain access tokens using the JWT bearer grant described